which correctly passes the first argument on to `foo` as an lvalue reference, while passing the others as rvalue references.

The final thing you can do with a parameter pack is find its size with the `sizeof...` operator. This is quite simple: `sizeof...(p)` is the number of elements in the parameter pack `p`. It doesn't matter whether this is a type parameter pack or a function argument parameter pack; the result is the same. This is probably the only case where you can use a parameter pack and not follow it with an ellipsis; the ellipsis is already part of the `sizeof...` operator. The following function returns the number of arguments supplied to it:

```
template<typename ... Args>
unsigned count_args(Args ... args)
{
    return sizeof... (Args);
}
```

Just as with the normal `sizeof` operator, the result of `sizeof...` is a constant expression, so it can be used for specifying array bounds and so forth.

A.7. Automatically deducing the type of a variable

C++ is a statically typed language: the type of every variable is known at compile time. Not only that, but as a programmer you have to actually specify the type of each variable. In some cases this can lead to quite unwieldy names, for example:

```
std::map<std::string,std::unique_ptr<some_data>> m;
std::map<std::string,std::unique_ptr<some_data>>::iterator
    iter=m.find("my key");
```

Traditionally, the solution has been to use `typedef`s to reduce the length of a type identifier and potentially eliminate problems due to inconsistent types. This still works in C++11, but there's now a new way: if a variable is initialized in its declaration from a value of the same type, then you can specify the type as `auto`. In this case, the compiler will automatically deduce